머신러닝

2주차.

Scikit-Learn을 이용한 타이타닉 생존자 예측모델 생성 및 학습

학습 목표

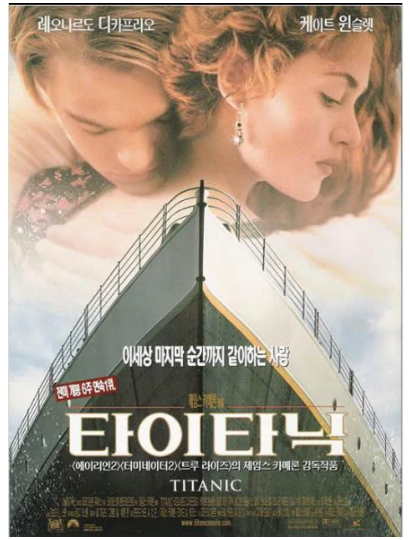
- **Titanic 데이터 셋 소개**

- **Titanic 타이타닉 생존자 예측모델**
  - 데이터 리뷰
  - 데이터 분석
  - 데이터 전처리(결측값, 데이터 인코딩, 삭제)
  - 모델 선택 및 학습
  - 학습 및 평가

Titanic 데이터 셋 소개

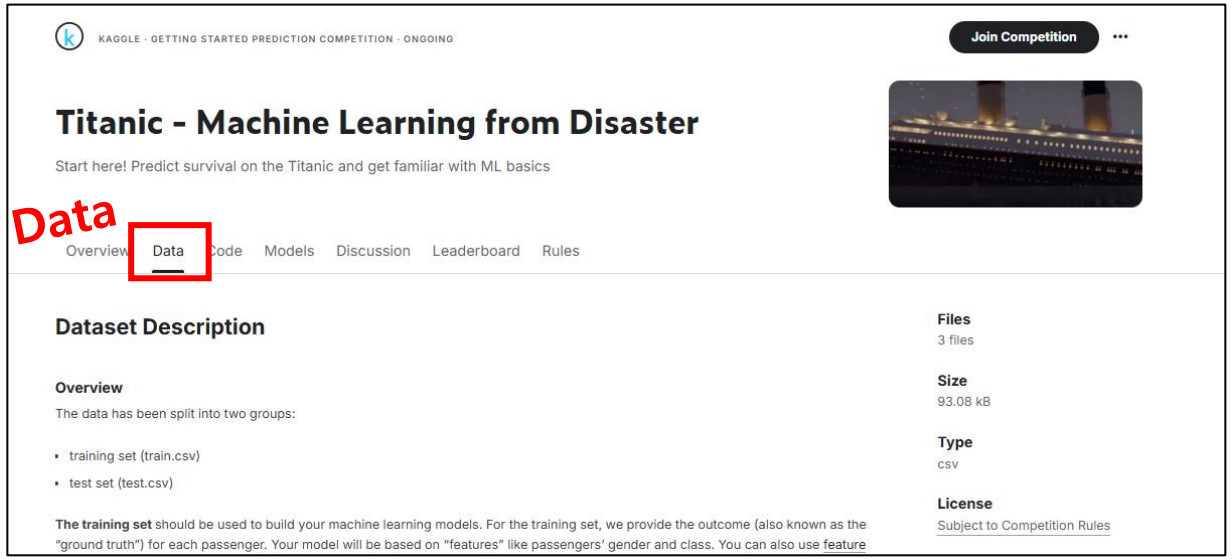
- **Kaggle의 Titanic 데이터셋**
  - 입문자들이 데이터 분석과 머신러닝을 배우기 위해 널리 사용되는 데이터셋 중 하나
  - 1912년에 발생한 타이타닉 호 침몰 사고에서의 승객들의 생존 정보를 포함
    - 891개의 승객의 나이, 성별, 티켓 등급(1등석, 2등석, 3등석), 탑승 항구 등 다양한 특성(Feature)을 포함

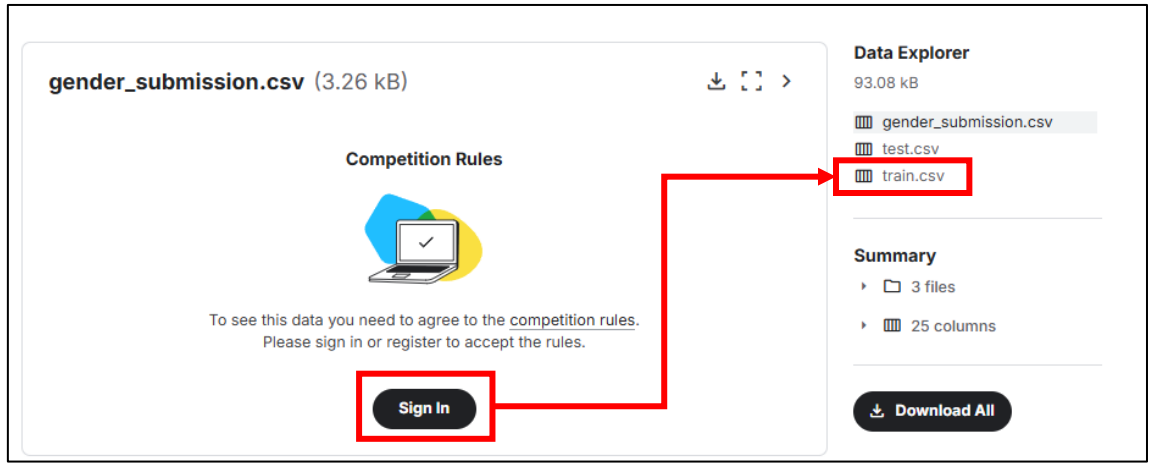
- **데이터 분석**
  - 승객의 정보(나이, 성별, 선실 등)를 바탕으로 **생존 여부(Survived)**를 예측
  - Sex, Age, Pclass: 생존률에 큰 영향을 줌
  - SibSp, Parch: 가족 동반 여부가 생존률에 영향
  - Fare: 경제적 수준의 대리 변수
  - Embarked: 탑승한 항구에 따른 생존률 차이 분석 가능



Titanic 데이터 셋 다운 방법

- **download link**
  - 제공 웹사이트: https://www.kaggle.com/c/titanic
    - 로그인 필요

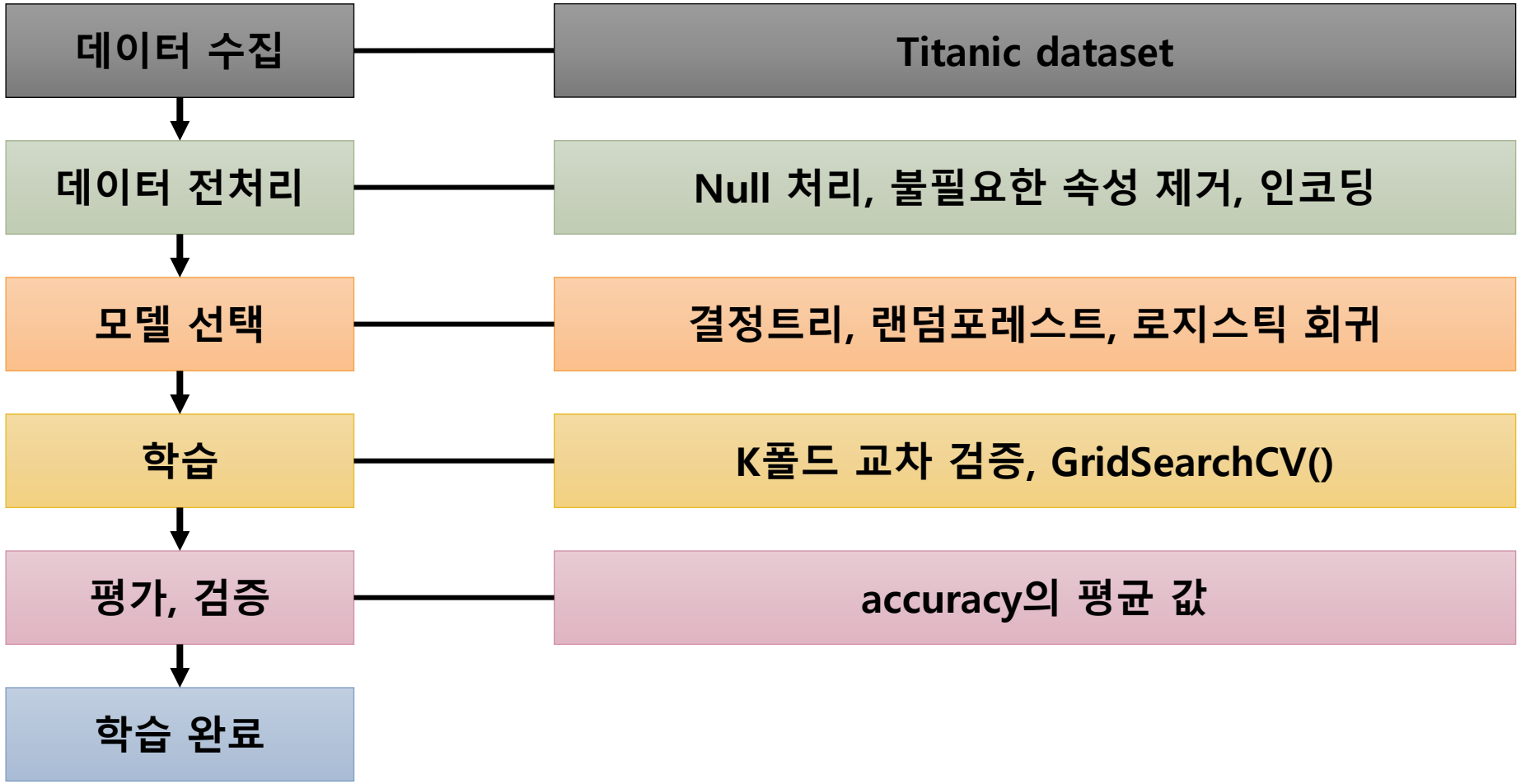




tarin.csv 파일 다운

- https://raw.githubusercontent.com/visualpython/visualpython/main/visualpython/data/sample_csv/titanic.csv

타이타닉 생존자 예측 모델 구현 순서



타이타닉 생존자 예측 모델 구현

- 데이터 로드 및 패키지 import

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline

titanic_df = pd.read_csv('./titanic_train.csv')
titanic_df.head(3)
```

데이터 시각화 라이브러리

설치 안되었을경우
가상환경에 설치

conda install matplotlib, seaborn

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S

타이타닉 생존자 예측 모델 구현
- 타이타닉 데이터 review

컬럼명	설명	데이터 유형	예시
PassengerId	각 승객에게 주어진 고유 ID 번호	정수형	1,2,3,...
Survived (target)	생존유무, target 값	이진형(0 = 사망, 1 = 생존)	0, 1
Pclass	객실 클래스(1 = 1등석, 2 = 2등석, 3 = 3등석)	범주형	1,2,3
Name	탑승객 성명	문자열	"Braund, Mr. Owen Harris"
Sex	성별	범주형	"male", "female"
Age	나이(세)	실수형	22.0, 38.0
SibSp	동승한 형제자매, 배우자 수	정수형	1, 0
Parch	동승한 부모, 자녀 수	정수형	0, 2
Ticket	티켓 넘버	문자열	"A/5 21171"
Fare	탑승 요금	실수형	7.25, 71.2833
Cabin	객실 넘버	문자열 (결측 다수 존재)	"C85"
Embarked	탑승 항구 (C = Cherbourg, Q = Queenstown, S = Southampton)	범주형	"S"

타이타닉 생존자 예측 모델 구현
- 타이타닉 데이터 review(계속)

- **Titanic 데이터 정보**

```
print('\n ### train 데이터 정보 ###  \n')
print(titanic_df.info())
```

- 전체 데이터의 수: 891건

- Null 값이 존재
  - Age, Cabin, Embarked
  - fillna() 함수로 값 대체

- Object Type(문자열)
  - Name, Sex, Ticket, Cabin, Embarked
  - 레이블 인코딩 또는 삭제 처리

```
 ### train 데이터 정보 ###

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
None
```

문자열 확인

Null 값 확인

타이타닉 생존자 예측 모델 구현
- 타이타닉 데이터 review(계속)

- **Titanic 수치형 데이터 정보**

```
print('\n ### train 수치형 데이터 정보 ###  \n')
titanic_df.describe()
```

- count: 전체 데이터 수
- mean: 평균
- std: 표준편차
- min: 최소값
- 25%: 1사분위수
- 50%: 2사분위수
- 75%: 3사분위수
- max: 최대값

```
### train 수치형 데이터 정보 ###
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

타이타닉 생존자 예측 모델 구현
- 타이타닉 데이터 분석

- **성별('Sex')와 생존('Survived)의 관계**
  - groupby: 주어진 컬럼을 묶어줌 연산을 수행
    - 'Sex'와 'Survived"를 묶어서 'Sex'의 값에 따라 Survived의 컬럼 값을 카운팅
  - 'Survived'의 레이블링 값
    - 0: 사망
    - 1: 생존

```
titanic_df.groupby(['Sex','Survived'])['Survived'].count()
```

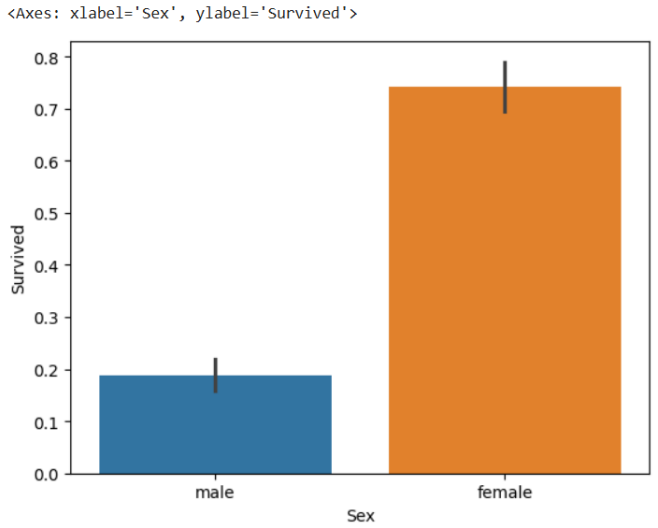
```
Sex     Survived
female  0            81
        1           233
male    0           468
        1           109
Name: Survived, dtype: int64
```

여성(female)이 남성(male)보다 상대적으로 많이 생존(Survived: 1)

타이타닉 생존자 예측 모델 구현
- 타이타닉 데이터 분석(계속)

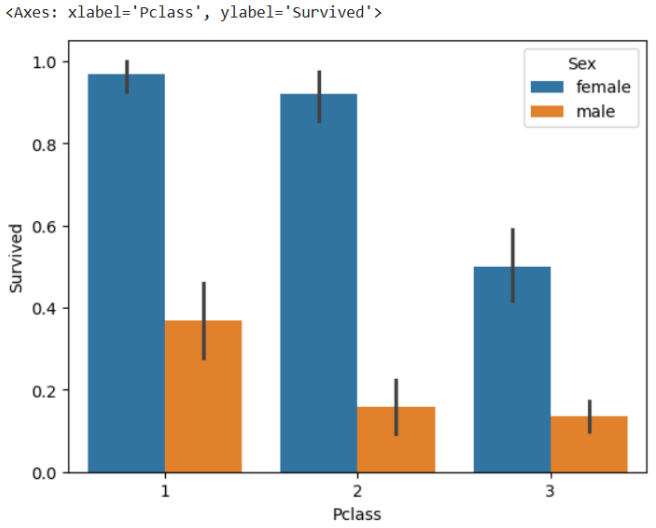
- **생존('Survived)의 시각화**
  - sns.barplot()그래프: 주어진 값들의 평균값을 시각화
    - 입력값: x축 이름, y축 이름, 입력 데이터

```
sns.barplot(x='Sex', y = 'Survived', hue = 'Sex', data=titanic_df)
```



남성(male)보다 상대적으로 여성(female)이 많이 생존(Survived: 1)

```
sns.barplot(x='Pclass', y='Survived', hue='Sex', data=titanic_df)
```



등급('Pclass')이 1등급일때 남자가 조금더 생존

타이타닉 생존자 예측 모델 구현
- 타이타닉 데이터 분석(계속)

- **나이('Age')와 생존(Survived)의 상관 관계**
  - 승객 나이에 따라 카테고리 분류
  - 'Unkown', 'Baby', 'Child', 'Teenager', 'Student', 'Young Adult', 'Adult', 'Elderly'

```
# 입력 age에 따라 구분값을 반환하는 함수 설정. DataFrame의 apply lambda식에 사용.
def get_category(age):
    cat = ''
    if age <= -1: cat = 'Unknown'
    elif age <= 5: cat = 'Baby'
    elif age <= 12: cat = 'Child'
    elif age <= 18: cat = 'Teenager'
    elif age <= 25: cat = 'Student'
    elif age <= 35: cat = 'Young Adult'
    elif age <= 60: cat = 'Adult'
    else : cat = 'Elderly'

    return cat
```

타이타닉 생존자 예측 모델 구현
- 타이타닉 데이터 분석(계속)

- **나이('Age')와 생존(Survived)의 상관 관계(계속)**
  - df.apply(lambda 식): DataFrame의 컬럼값을 lambda 식에 적용

```
# 막대그래프의 크기 figure를 더 크게 설정
plt.figure(figsize=(10,6))

#X축의 값을 순차적으로 표시하기 위한 설정
group_names = ['Unknown', 'Baby', 'Child', 'Teenager', 'Student', 'Young Adult', 'Adult', 'Elderly']

# lambda 식에 위에서 생성한 get_category( ) 함수를 반환값으로 지정.
# get_category(X)는 입력값으로 'Age' 컬럼값을 받아서 해당하는 cat 반환
titanic_df['Age_cat'] = titanic_df['Age'].apply(lambda x : get_category(x))
```

```
titanic_df['Age']

0      22.000000
1      38.000000
2      26.000000
3      35.000000
4      35.000000
         ...
886    27.000000
887    19.000000
888    29.699118
889    26.000000
890    32.000000
Name: Age, Length: 891, dtype: float64
```

```
Student
Adult
Young Adult
Young Adult
Young Adult
...
Young Adult
Student
Young Adult
Young Adult
Young Adult
```

titanic_df['Age'] 컬럼의 값을
lamda x : get_category(x)에 적용

반환값으로
titanic_df['Age_cat'] 컬럼 생성

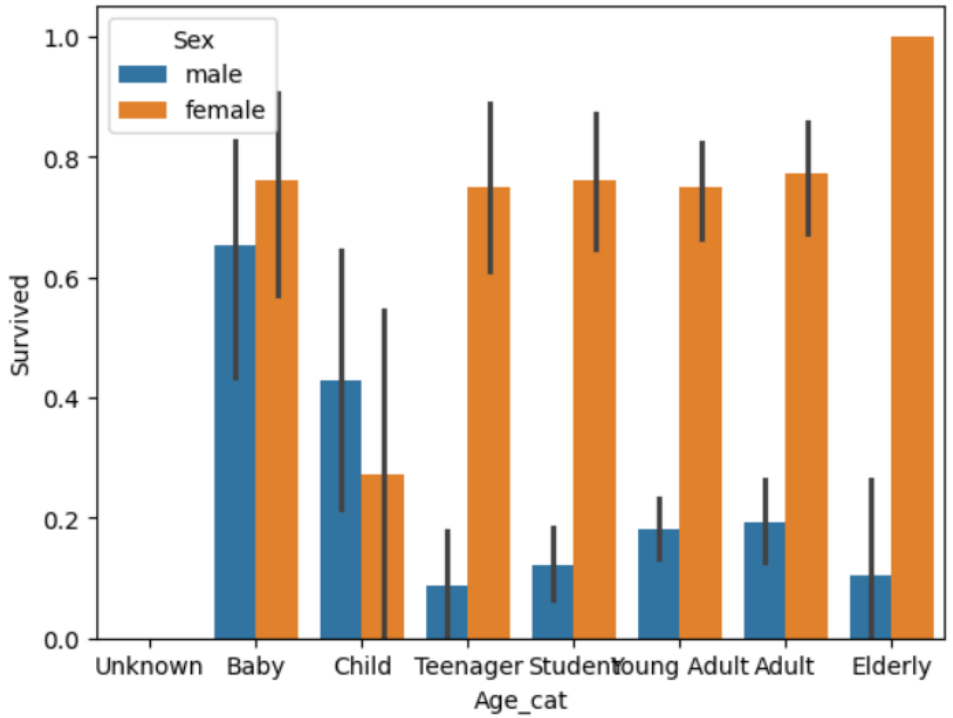
```
titanic_df['Age_cat']

0          Student
1            Adult
2      Young Adult
3      Young Adult
4      Young Adult
          ...
886    Young Adult
887        Student
888    Young Adult
889    Young Adult
890    Young Adult
Name: Age_cat, Length: 891, dtype: object
```

타이타닉 생존자 예측 모델 구현
- 타이타닉 데이터 분석(계속)

- 나이('Age')와 생존(Survived)의 상관 관계(계속)

```
sns.barplot(x='Age_cat', y = 'Survived', hue='Sex', data=titanic_df, order=group_names)
titanic_df.drop('Age_cat', axis=1, inplace=True)
```



여성(female)의 생존율이 높음

어린(Baby) 남자(male)의
생존율이 높음

타이타닉 생존자 예측 모델 구현
- 타이타닉 데이터 전처리 – 결측값 처리

- **NULL 컬럼들에 대한 처리**
  - fillna() 메소드 사용
    - Age: 평균 값으로 대체
    - Cabin, Embarked: 'N'(None)으로 대체
  - inplace = True : 해당 데이터프레임에 바로 업데이트

```
titanic_df['Age'].fillna(titanic_df['Age'].mean(),inplace=True)
titanic_df['Cabin'].fillna('N',inplace=True)
titanic_df['Embarked'].fillna('N',inplace=True)
print('데이터 세트 Null 값 갯수 ',titanic_df.isnull().sum().sum())
```

평균 값 으로 대체

'N'으로 대체

```
데이터 세트 Null 값 갯수  0
```

titanic_df.isnull().sum()
/ 각 컬럼의 null값의 총합

```
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
Cabin          0
Embarked       0
dtype: int64
```

null값의 총합

타이타닉 생존자 예측 모델 구현
- 타이타닉 데이터 전처리 – 결측값 처리(계속)

- **Titanic 데이터**

```
print(' Sex 값 분포 :\n',titanic_df['Sex'].value_counts())
print('\n Cabin 값 분포 :\n',titanic_df['Cabin'].value_counts())
print('\n Embarked 값 분포 :\n',titanic_df['Embarked'].value_counts())
```

```
 Sex 값 분포 :
 Sex
male      577
female    314
Name: count, dtype: int64
```

```
 Cabin 값 분포 :
 Cabin
N              687
C23 C25 C27      4
G6               4
B96 B98          4
C22 C26          3
                ...
E34              1
C7               1
C54              1
E36              1
C148             1
Name: count, Length: 148, dtype: int64
```

```
 Embarked 값 분포 :
 Embarked
S    644
C    168
Q     77
N      2
Name: count, dtype: int64
```

타이타닉 생존자 예측 모델 구현
- 타이타닉 데이터 전처리 – 데이터 인코딩

- 'Cabin' 컬럼 값 단순화

```
titanic_df['Cabin'] = titanic_df['Cabin'].str[:1]
print(titanic_df['Cabin'].head(3))
```

```
0    N
1    C
2    N
Name: Cabin, dtype: object
```

'Cabin' 컬럼의 문자열의 앞에 한글자만[:1] 추출

titanic_df['Cabin']

```
0        N
1       C85
2        N
3      C123
4        N
       ...
886      N
887    B42
888      N
889    C148
890      N
Name: Cabin, Length: 891, dtype: object
```

단순화

titanic_df['Cabin'].str[:1]

```
0      N
1      C
2      N
3      C
4      N
      ..
886    N
887    B
888    N
889    C
890    N
Name: Cabin, Length: 891, dtype: object
```

단순화된 'Cabin' 값

titanic_df['Cabin'].value_counts()

```
Cabin
N    687
C     59
B     47
D     33
E     32
A     15
F     13
G      4
T      1
Name: count, dtype: int64
```

타이타닉 생존자 예측 모델 구현
- 타이타닉 데이터 전처리 – 데이터 인코딩(계속)

- **문자열(NaN)인 컬럼값의 인코딩**
  - 'Cabin', 'Sex', 'Embarked'의 값을 레이블 인코딩 적용

```
from sklearn import preprocessing

def encode_features(dataDF):
    features = ['Cabin', 'Sex', 'Embarked']
    for feature in features:
        le = preprocessing.LabelEncoder()
        le = le.fit(dataDF[feature])
        dataDF[feature] = le.transform(dataDF[feature])

    return dataDF

titanic_df = encode_features(titanic_df)
titanic_df.head()
```

LabelEncoder 객체 생성
fit() / transform() 변환

타이타닉 생존자 예측 모델 구현
- 타이타닉 데이터 전처리 – 데이터 인코딩(계속)

- 레이블 인코딩 결과

레이블 인코딩 전

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	N	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	N	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	N	S

레이블 인코딩 후

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	1	22.0	1	0	A/5 21171	7.2500	7	3
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	0	38.0	1	0	PC 17599	71.2833	2	0
2	3	1	3	Heikkinen, Miss. Laina	0	26.0	0	0	STON/O2. 3101282	7.9250	7	3
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	0	35.0	1	0	113803	53.1000	2	3
4	5	0	3	Allen, Mr. William Henry	1	35.0	0	0	373450	8.0500	7	3

타이타닉 생존자 예측 모델 구현
- 타이타닉 데이터 전처리 – 데이터 삭제

- **학습에 사용되지 않는 컬럼값 삭제**
  - drop(): 데이터 삭제
    - axis = 1(컬럼) 삭제

  - 삭제 컬럼 값
    - 'PassengerID', 'Name', 'Ticket'

```
def drop_features(df):
    df.drop(['PassengerId', 'Name', 'Ticket'], axis=1, inplace=True)
    return df
```

타이타닉 생존자 예측 모델 구현
- 타이타닉 데이터 전처리 - 정리

- **데이터 전처리 함수 생성**

```python
from sklearn.preprocessing import LabelEncoder

# Null 처리 함수
def fillna(df):
    df['Age'].fillna(df['Age'].mean(), inplace=True)
    df['Cabin'].fillna('N', inplace=True)
    df['Embarked'].fillna('N', inplace=True)
    df['Fare'].fillna(0, inplace=True)
    return df

# 머신러닝 알고리즘에 불필요한 피처 제거
def drop_features(df):
    df.drop(['PassengerId', 'Name', 'Ticket'], axis=1, inplace=True)
    return df


# 레이블 인코딩 수행.
def format_features(df):
    df['Cabin'] = df['Cabin'].str[:1]
    features = ['Cabin', 'Sex', 'Embarked']
    for feature in features:
        le = LabelEncoder()
        le = le.fit(df[feature])
        df[feature] = le.transform(df[feature])
    return df
```

```python
# 앞에서 설정한 데이터 전처리 함수 호출
def transform_features(df):
    df = fillna(df)
    df = drop_features(df)
    df = format_features(df)
    return df
```

fillna() 함수호출: 결측치 대체
drop_features(): 학습에 사용되지 않는 컬럼 삭제
format_features(): object(문자열)자료형, 수치형으로 인코딩

타이타닉 생존자 예측 모델 구현
- 타이타닉 데이터 – 피처/타겟 데이터

- 학습에 사용될 피처데이터 / 타켓 데이터

컬럼명	설명	데이터 유형
Pclass	객실 클래스(1 = 1등석, 2 = 2등석, 3 = 3등석)	범주형
Sex	성별	범주형
Age	나이(세)	실수형
SibSp	동승한 형제자매, 배우자 수	정수형
Parch	동승한 부모, 자녀 수	정수형
Fare	탑승 요금	실수형
Cabin	객실 넘버	문자열 (결측 다수 존재)
Embarked	탑승 항구 (C = Cherbourg, Q = Queenstown, S = Southampton)	범주형
Survived (target)	생존유무, target 값	이진형(0 = 사망, 1 = 생존)

타이타닉 생존자 예측 모델 구현
- 타이타닉 데이터 – 피처/타겟 데이터

- 피처 데이터 / 타겟(레이블) 데이터 분리

```
# 원본 데이터에 feature데이터 셋과 Label 데이터 셋 추출.
titanic_df = pd.read_csv('./titanic_train.csv')

y_titanic_df = titanic_df['Survived']

X_titanic_df= titanic_df.drop('Survived',axis=1)

X_titanic_df = transform_features(X_titanic_df)
```

원본 데이터 가져오기

'Survived' 컬럼만 빼서 타겟 데이터(y_titanic_df)로 분리

원본 데이터에서 'Survived' 컬럼만 삭제해서 피처 데이터(X_titanic_df)로 만들기

전처리 함수를 통해 피처 데이터 전처리

피처 데이터(X_titanic_df)

	Pclass	Sex	Age	SibSp	Parch	Fare	Cabin	Embarked
0	3	1	22.0	1	0	7.2500	7	3
1	1	0	38.0	1	0	71.2833	2	0
2	3	0	26.0	0	0	7.9250	7	3

타겟 데이터(y_titanic_df)

```
0    0
1    1
2    1
Name: Survived, dtype: int64
```

타이타닉 생존자 예측 모델 구현
- 타이타닉 데이터 – 학습/테스트 데이터 분리 및 학습 알고리즘

- 학습 / 테스트 데이터 분리

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test=train_test_split(X_titanic_df, y_titanic_df, \
                                                  test_size=0.2, random_state=11)
```

- 학습 알고리즘 import 및 객체 생성

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# 결정트리, Random Forest, 로지스틱 회귀를 위한 사이킷런 Classifier 클래스 생성
dt_clf = DecisionTreeClassifier(random_state=11)
rf_clf = RandomForestClassifier(random_state=11)
lr_clf = LogisticRegression(solver='liblinear')
```

타이타닉 생존자 예측 모델 구현
- 타이타닉 데이터 – 학습 및 평가

- **학습 및 평가**

```
# DecisionTreeClassifier 학습/예측/평가
dt_clf.fit(X_train , y_train)
dt_pred = dt_clf.predict(X_test)
print('DecisionTreeClassifier 정확도: {0:.4f}'.format(accuracy_score(y_test, dt_pred)))

# RandomForestClassifier 학습/예측/평가
rf_clf.fit(X_train , y_train)
rf_pred = rf_clf.predict(X_test)
print('RandomForestClassifier 정확도:{0:.4f}'.format(accuracy_score(y_test, rf_pred)))

# LogisticRegression 학습/예측/평가
lr_clf.fit(X_train , y_train)
lr_pred = lr_clf.predict(X_test)
print('LogisticRegression 정확도: {0:.4f}'.format(accuracy_score(y_test, lr_pred)))
```

```
DecisionTreeClassifier 정확도: 0.7877
RandomForestClassifier 정확도:0.8547
LogisticRegression 정확도: 0.8659
```



강남대학교

KANGNAM UNIVERSITY